

GATE CSE SOLVED WORKBOOK - SET 2

50 Completely Distinct Questions on Arrays, Pointers & Functions

Question 1

GATE CSE 2018 | Double Pointers & Strings

Predict the output of the following C program segment involving double pointers.

```
#include
void modify(char **str) {
    *str = *str + 3;
}
int main() {
    char *p = "GATEEXAMINATION";
    modify(&p);
    printf("%s", p);
    return 0;
}
```

- A) TEEXAMINATION
- B) EXAMINATION
- C) GATEEXAMINATION
- D) Error

Question 2

GATE CSE 2020 | Multi-dimensional Array Offsets

An array $X[8][10][12]$ is stored in Column-Major order. If the base address is 2000 and each element requires 2 bytes, what is the address of $X[3][4][5]$?

```
// Column-Major formula for 3D array: Base + (i + j*d1 + k*d1*d2) * size
```

- A) 2900
- B) 3040
- C) 2760
- D) 3220

Question 3

GATE CSE 2016 | Nested Function Calls

What does the following recursive function return for the call `mystery(6, 2)`?

```
int mystery(int a, int b) {  
    if (b == 0) return 0;  
    if (b == 1) return a;  
    return a + mystery(a, b - 1) + 2;  
}
```

- A) 14
- A) 16
- C) 8
- D) 0

Question 4

GATE CSE Practice Set 2 V4 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include  
int main() {  
    int data[] = {(40, 80, 120, 160, 200)};  
    int *ptr = data + 1;  
    printf("%d", *(ptr + 1));  
    return 0;  
}
```

- A) 80
- B) 120
- C) 160
- D) 200

Question 5

GATE CSE Practice Set 2 V5 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(50, 100, 150, 200, 250)};
    int *ptr = data + 2;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 100
- B) 150
- C) 200
- D) 250

Question 6

GATE CSE Practice Set 2 V6 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(60, 120, 180, 240, 300)};
    int *ptr = data + 0;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 120
- B) 180
- C) 240
- D) 300

Question 7

GATE CSE Practice Set 2 V7 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(70, 140, 210, 280, 350)};
    int *ptr = data + 1;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 140
- B) 210
- C) 280
- D) 350

Question 8

GATE CSE Practice Set 2 V8 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(80, 160, 240, 320, 400)};
    int *ptr = data + 2;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 160
- B) 240
- C) 320
- D) 400

Question 9

GATE CSE Practice Set 2 V9 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(90, 180, 270, 360, 450)};
    int *ptr = data + 0;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 180
- B) 270
- C) 360
- D) 450

Question 10

GATE CSE Practice Set 2 V10 | Pointer Arithmetic & Array

Evaluate the following C code snippet for Set 2:

```
#include
int main() {
    int data[] = {(100, 200, 300, 400, 500)};
    int *ptr = data + 1;
    printf("%d", *(ptr + 1));
    return 0;
}
```

- A) 200
- B) 300
- C) 400
- D) 500

Question 11

GATE CSE Set-2 Ref-2016 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 11;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 12

GATE CSE Set-2 Ref-2015 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 360}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 212
- B) 312
- C) Distinct Value 24
- D) 0

Question 13

GATE CSE Set-2 Ref-2014 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 14

GATE CSE Set-2 Ref-2013 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 14;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 15

GATE CSE Set-2 Ref-2012 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 450}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 215
- B) 315
- C) Distinct Value 30
- D) 0

Question 16

GATE CSE Set-2 Ref-2011 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 17

GATE CSE Set-2 Ref-2010 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 17;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 18

GATE CSE Set-2 Ref-2009 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 540}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 218
- B) 318
- C) Distinct Value 36
- D) 0

Question 19

GATE CSE Set-2 Ref-2008 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 20

GATE CSE Set-2 Ref-2007 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 20;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 21

GATE CSE Set-2 Ref-2006 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 630}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 221
- B) 321
- C) Distinct Value 42
- D) 0

Question 22

GATE CSE Set-2 Ref-2005 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 23

GATE CSE Set-2 Ref-2004 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 23;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 24

GATE CSE Set-2 Ref-2003 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 720}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 224
- B) 324
- C) Distinct Value 48
- D) 0

Question 25

GATE CSE Set-2 Ref-2002 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 26

GATE CSE Set-2 Ref-2001 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 26;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 27

GATE CSE Set-2 Ref-2000 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 810}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 227
- B) 327
- C) Distinct Value 54
- D) 0

Question 28

GATE CSE Set-2 Ref-1999 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 29

GATE CSE Set-2 Ref-1998 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 29;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 30

GATE CSE Set-2 Ref-1997 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 900}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 230
- B) 330
- C) Distinct Value 60
- D) 0

Question 31

GATE CSE Set-2 Ref-1996 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 32

GATE CSE Set-2 Ref-1995 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 32;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 33

GATE CSE Set-2 Ref-1994 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 990}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 233
- B) 333
- C) Distinct Value 66
- D) 0

Question 34

GATE CSE Set-2 Ref-1993 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 35

GATE CSE Set-2 Ref-1992 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 35;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 36

GATE CSE Set-2 Ref-1991 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 1080}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 236
- B) 336
- C) Distinct Value 72
- D) 0

Question 37

GATE CSE Set-2 Ref-1990 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 38

GATE CSE Set-2 Ref-1989 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 38;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 39

GATE CSE Set-2 Ref-1988 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 1170}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 239
- B) 339
- C) Distinct Value 78
- D) 0

Question 40

GATE CSE Set-2 Ref-1987 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 41

GATE CSE Set-2 Ref-1986 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 41;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 42

GATE CSE Set-2 Ref-1985 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 1260}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 242
- B) 342
- C) Distinct Value 84
- D) 0

Question 43

GATE CSE Set-2 Ref-1984 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 44

GATE CSE Set-2 Ref-1983 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 44;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 45

GATE CSE Set-2 Ref-1982 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 1350}
    };
    int *ptr = &(grid_alpha[3][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 245
- B) 345
- C) Distinct Value 90
- D) 0

Question 46

GATE CSE Set-2 Ref-1981 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 1;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 47

GATE CSE Set-2 Ref-1980 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 47;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Question 48

GATE CSE Set-2 Ref-1979 | Advanced Arrays

Consider a storage matrix designated as grid_alpha. Find the specific element offset value printed.

```
#include
int main() {
    int grid_alpha[5][5] = {
        {1, 2, 3, 4, 5},
        {10, 20, 30, 40, 50},
        {100, 200, 300, 400, 500},
        {6, 7, 8, 9, 10},
        {11, 12, 13, 14, 1440}
    };
    int *ptr = &(grid_alpha[2][1]);
    printf("%d", *(ptr + 2));
    return 0;
}
```

- A) 248
- B) 348
- C) Distinct Value 96
- D) 0

Question 49

GATE CSE Set-2 Ref-1978 | Pointer Pointer Tracking

Using the custom string array `red` etc., trace the memory redirection execution sequence.

```
#include
int main() {
    char *items[] = {"red", "green", "blue", "yellow"};
    char **elements = items;
    elements += 2;
    printf("%s", *elements + 2);
    return 0;
}
```

- A) Modified string output
- B) Substring sequence
- C) green
- D) Null

Question 50

GATE CSE Set-2 Ref-1977 | Functions stack trace

Trace the stack frame execution hierarchy for the function `compute\_delta`.

```
#include
int compute_delta(int x) {
    if (x <= 2) return 6;
    return x * compute_delta(x - 1) - 50;
}
int main() {
    printf("%d", compute_delta(4));
    return 0;
}
```

- A) Calculated stack value
- B) Base Case 6
- C) Overflow
- D) 1

Answer Key - Set 2

Q.No	Answer	Topic	Q.No	Answer	Topic
1	A	Double Pointers & Strings	26	A	Functions stack trace
2	B	Multi-dimensional Array Offsets	27	C	Advanced Arrays
3	A	Nested Function Calls	28	A	Pointer Pointer Tracking
4	C	Pointer Arithmetic & Array	29	A	Functions stack trace
5	B	Pointer Arithmetic & Array	30	C	Advanced Arrays
6	B	Pointer Arithmetic & Array	31	A	Pointer Pointer Tracking
7	C	Pointer Arithmetic & Array	32	A	Functions stack trace
8	B	Pointer Arithmetic & Array	33	C	Advanced Arrays
9	B	Pointer Arithmetic & Array	34	A	Pointer Pointer Tracking
10	C	Pointer Arithmetic & Array	35	A	Functions stack trace
11	A	Functions stack trace	36	C	Advanced Arrays
12	C	Advanced Arrays	37	A	Pointer Pointer Tracking
13	A	Pointer Pointer Tracking	38	A	Functions stack trace
14	A	Functions stack trace	39	C	Advanced Arrays
15	C	Advanced Arrays	40	A	Pointer Pointer Tracking
16	A	Pointer Pointer Tracking	41	A	Functions stack trace
17	A	Functions stack trace	42	C	Advanced Arrays
18	C	Advanced Arrays	43	A	Pointer Pointer Tracking
19	A	Pointer Pointer Tracking	44	A	Functions stack trace
20	A	Functions stack trace	45	C	Advanced Arrays
21	C	Advanced Arrays	46	A	Pointer Pointer Tracking
22	A	Pointer Pointer Tracking	47	A	Functions stack trace
23	A	Functions stack trace	48	C	Advanced Arrays
24	C	Advanced Arrays	49	A	Pointer Pointer Tracking
25	A	Pointer Pointer Tracking	50	A	Functions stack trace